

EE 604 Image Processing

Lec 03: Spatial Domain - II

Koteswar Rao Jerripothula
Associate Professor
EE Department
IIT Kanpur

Outline

IMAGE HISTOGRAMS

OTSU'S METHOD

HISTOGRAM EQUALIZATION

HISTOGRAM MATCHING

SPATIAL FILTERING

Histogram

Let r_k , for $k = 0, 1, 2, \dots, L - 1$, denote the intensities of an L -level digital image, $f(x, y)$. The *unnormalized histogram* of f is defined as

$$h(r_k) = n_k \quad \text{for } k = 0, 1, 2, \dots, L - 1$$

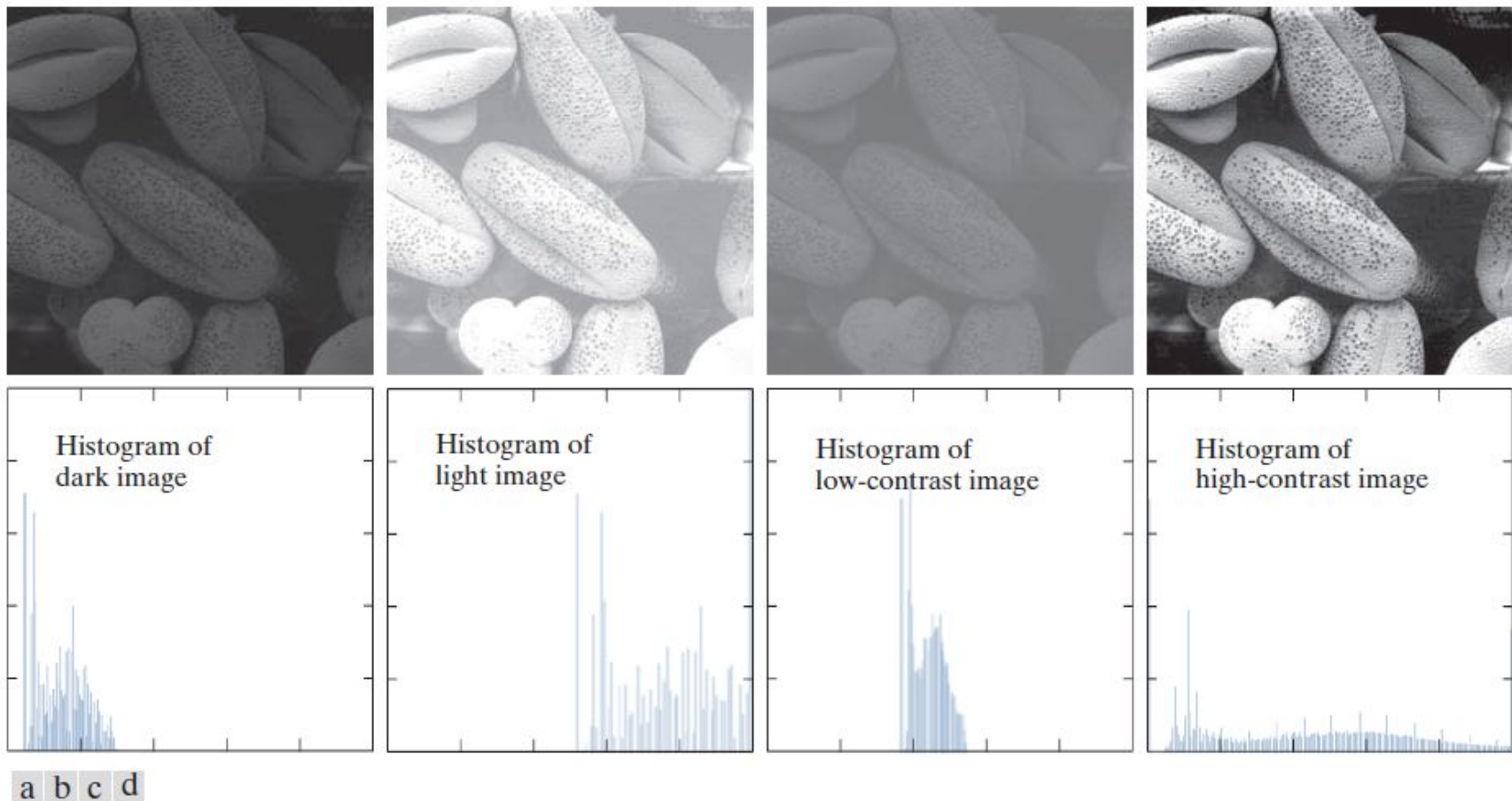
where n_k is the number of pixels in f with intensity r_k , and the subdivisions of the intensity scale are called *histogram bins*. Similarly, the *normalized histogram* of f is defined as

$$p(r_k) = \frac{h(r_k)}{MN} = \frac{n_k}{MN}$$

where, as usual, M and N are the number of image rows and columns, respectively.

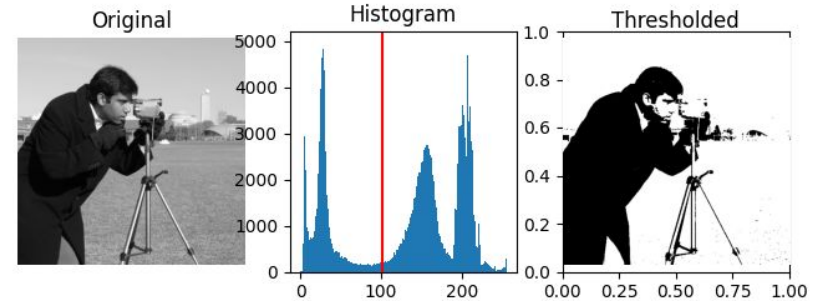
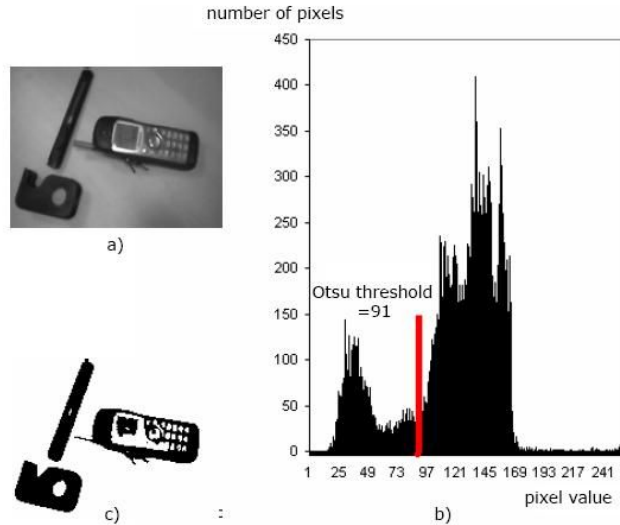
The sum of $p(r_k)$ for all values of k is always 1.

It gives us estimates of the probabilities of intensity levels occurring in an image.



Four image types and their corresponding histograms. (a) dark; (b) light; (c) low contrast; (d) high contrast. The horizontal axis of the histograms are values of r_k and the vertical axis are values of $p(r_k)$.

Otsu Thresholding



Otsu's method is a well-known technique used for automatic image thresholding. The main purpose is to determine an optimal threshold value that effectively separates pixels in an image into two distinct classes.

Otsu Thresholding

$$\sigma_w^2(T) = w_0(T)\sigma_0^2(T) + w_1(T)\sigma_1^2(T).$$

Intra Class
Variance

Considering the intensities to lie in $\{0, \dots, L - 1\}$ and h the histogram, the variables are defined as below.

Class 0

$$w_0(T) = \sum_{i=0}^T h(i)$$

$$\mu_0(T) = \frac{1}{w_0(T)} \sum_{i=0}^T ih(i)$$

$$\sigma_0^2(T) = \frac{1}{w_0(T)} \sum_{i=0}^T (i - \mu_0(T))^2 h(i)$$

Weight

Weighted Mean of Intensities

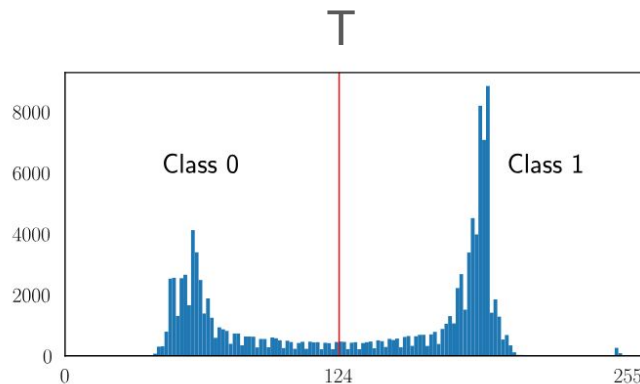
Weighted Variance of Intensities

Class 1

$$w_1(T) = \sum_{i=T+1}^{L-1} h(i)$$

$$\mu_1(T) = \frac{1}{w_1(T)} \sum_{i=T+1}^{L-1} ih(i)$$

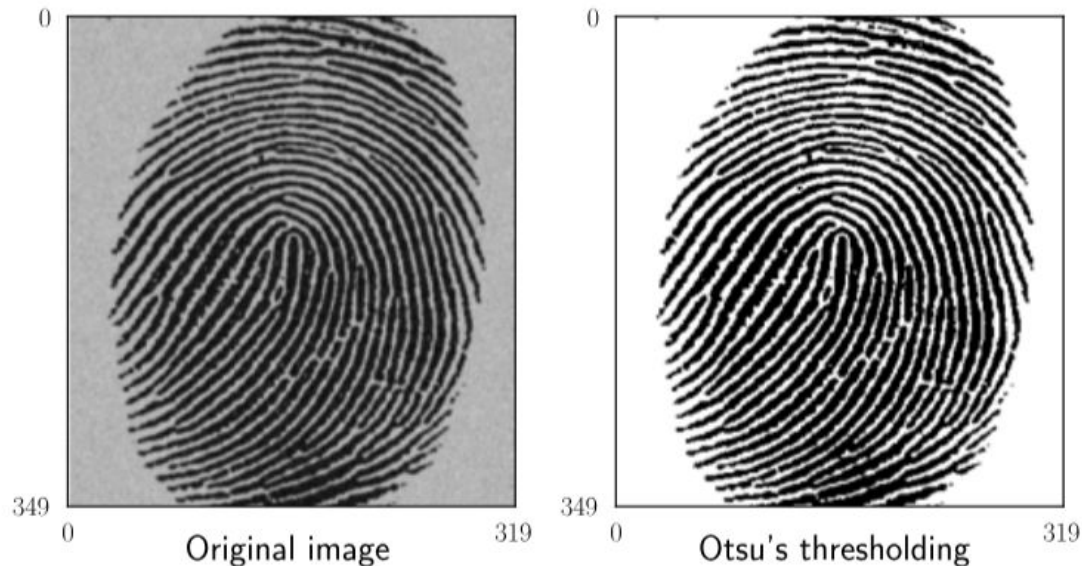
$$\sigma_1^2(T) = \frac{1}{w_1(T)} \sum_{i=T+1}^{L-1} (i - \mu_1(T))^2 h(i)$$



An histogram and a threshold T .

The algorithm to determine the value of T that minimize $\sigma_w^2(T)$ is simple: the intra-class variance $\sigma_w^2(T)$ is calculated for all the thresholds $T = \{0, \dots, L - 1\}$, and the one that gives the lowest value for $\sigma_w^2(T)$ is returned.

An example of Otsu's thresholding





Original Image



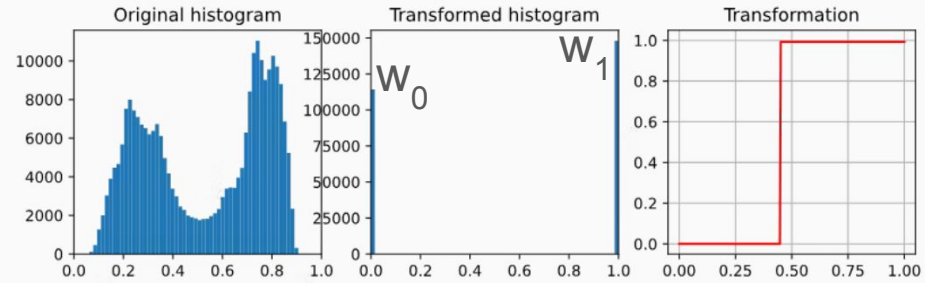
Thresholded and segmented Image



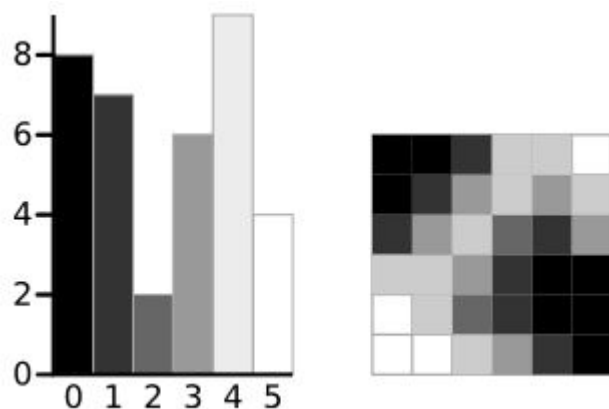
Original image



Transformed image

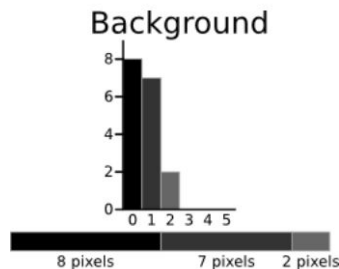


Histogram transformations



A 6-level greyscale image and its histogram

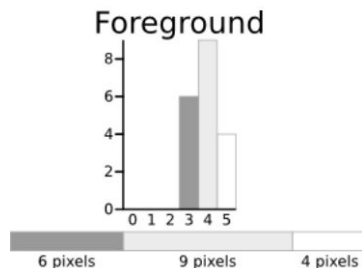
@ T=2



$$\text{Weight } W_b = \frac{8 + 7 + 2}{36} = 0.4722$$

$$\text{Mean } \mu_b = \frac{(0 \times 8) + (1 \times 7) + (2 \times 2)}{17} = 0.6471$$

$$\begin{aligned} \text{Variance } \sigma_b^2 &= \frac{((0 - 0.6471)^2 \times 8) + ((1 - 0.6471)^2 \times 7) + ((2 - 0.6471)^2 \times 2)}{17} \\ &= \frac{(0.4187 \times 8) + (0.1246 \times 7) + (1.8304 \times 2)}{17} \\ &= 0.4637 \end{aligned}$$



$$\text{Weight } W_f = \frac{6 + 9 + 4}{36} = 0.5278$$

$$\text{Mean } \mu_f = \frac{(3 \times 6) + (4 \times 9) + (5 \times 4)}{19} = 3.8947$$

$$\begin{aligned} \text{Variance } \sigma_f^2 &= \frac{((3 - 3.8947)^2 \times 6) + ((4 - 3.8947)^2 \times 9) + ((5 - 3.8947)^2 \times 4)}{19} \\ &= \frac{(4.8033 \times 6) + (0.0997 \times 9) + (4.8864 \times 4)}{19} \\ &= 0.5152 \end{aligned}$$

The next step is to calculate the 'Within-Class Variance'. This is simply the sum of the two variances multiplied by their associated weights.

$$\begin{aligned} \text{Within Class Variance } \sigma_W^2 &= W_b \sigma_b^2 + W_f \sigma_f^2 = 0.4722 * 0.4637 + 0.5278 * 0.5152 \\ &= 0.4909 \end{aligned}$$

By a bit of manipulation, you can calculate what is called the *between class* variance, which is far quicker to calculate. Luckily, the threshold with the maximum *between class* variance also has the minimum *within class* variance. So it can also be used for finding the best threshold and therefore due to being simpler is a much better approach to use.

$$\begin{aligned}\text{Within Class Variance} \quad \sigma_W^2 &= W_b \sigma_b^2 + W_f \sigma_f^2 \quad (\text{as seen above}) \\ \text{Between Class Variance} \quad \sigma_B^2 &= \sigma^2 - \sigma_W^2 \\ &= W_b (\mu_b - \mu)^2 + W_f (\mu_f - \mu)^2 \quad (\text{where } \mu = W_b \mu_b + W_f \mu_f) \\ &= W_b W_f (\mu_b - \mu_f)^2\end{aligned}$$

- **Weight identity:**

$$\omega_b + \omega_f = 1$$

- **Global mean identity:**

$$\mu = \omega_b \mu_b + \omega_f \mu_f$$

$$\sigma_B^2 = \omega_b(\mu_b - \mu)^2 + \omega_f(\mu_f - \mu)^2$$

$$\mu_b - \mu = \mu_b - (\omega_b\mu_b + \omega_f\mu_f)$$

$$\mu_b - \mu = \mu_b(1 - \omega_b) - \omega_f\mu_f$$

Apply the weight identity constraint $1 - \omega_b = \omega_f$:

$$\mu_b - \mu = \mu_b\omega_f - \omega_f\mu_f = \omega_f(\mu_b - \mu_f)$$

$$\mu_f - \mu = \mu_f - (\omega_b\mu_b + \omega_f\mu_f)$$

$$\mu_f - \mu = \mu_f(1 - \omega_f) - \omega_b\mu_b$$

Apply the weight identity constraint $1 - \omega_f = \omega_b$:

$$\mu_f - \mu = \mu_f\omega_b - \omega_b\mu_b = -\omega_b(\mu_b - \mu_f)$$

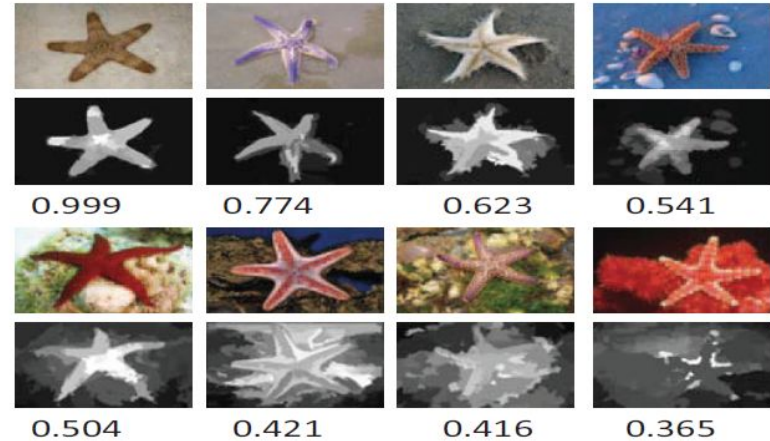
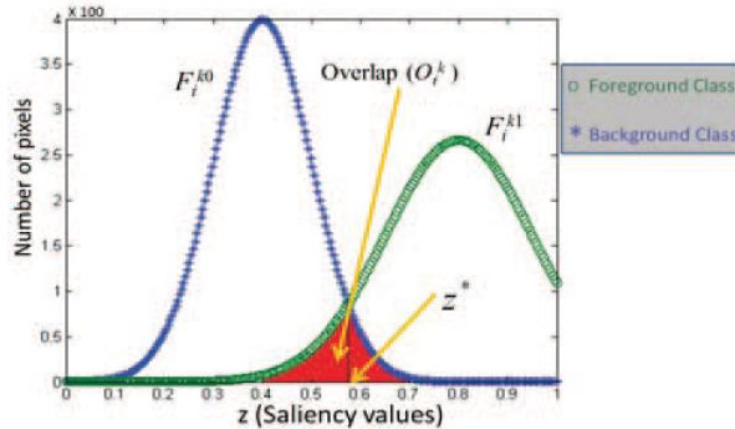
$$\sigma_B^2 = \omega_b [\omega_f(\mu_b - \mu_f)]^2 + \omega_f [-\omega_b(\mu_b - \mu_f)]^2$$

$$\sigma_B^2 = \omega_b\omega_f^2(\mu_b - \mu_f)^2 + \omega_f\omega_b^2(\mu_b - \mu_f)^2$$

$$\sigma_B^2 = \omega_b\omega_f(\mu_b - \mu_f)^2(\omega_f + \omega_b)$$

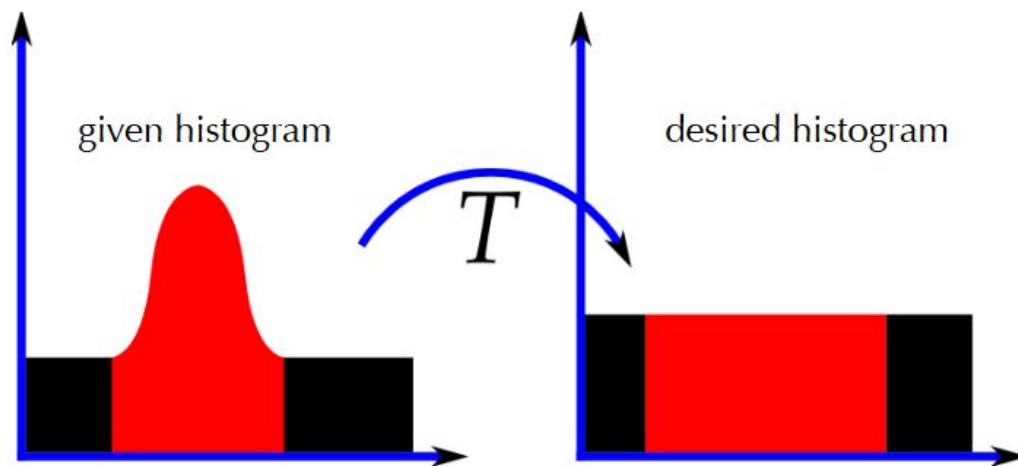
$$\sigma_B^2 = \omega_b\omega_f(\mu_b - \mu_f)^2$$

Application of Otsu Algorithm in Saliency Quality



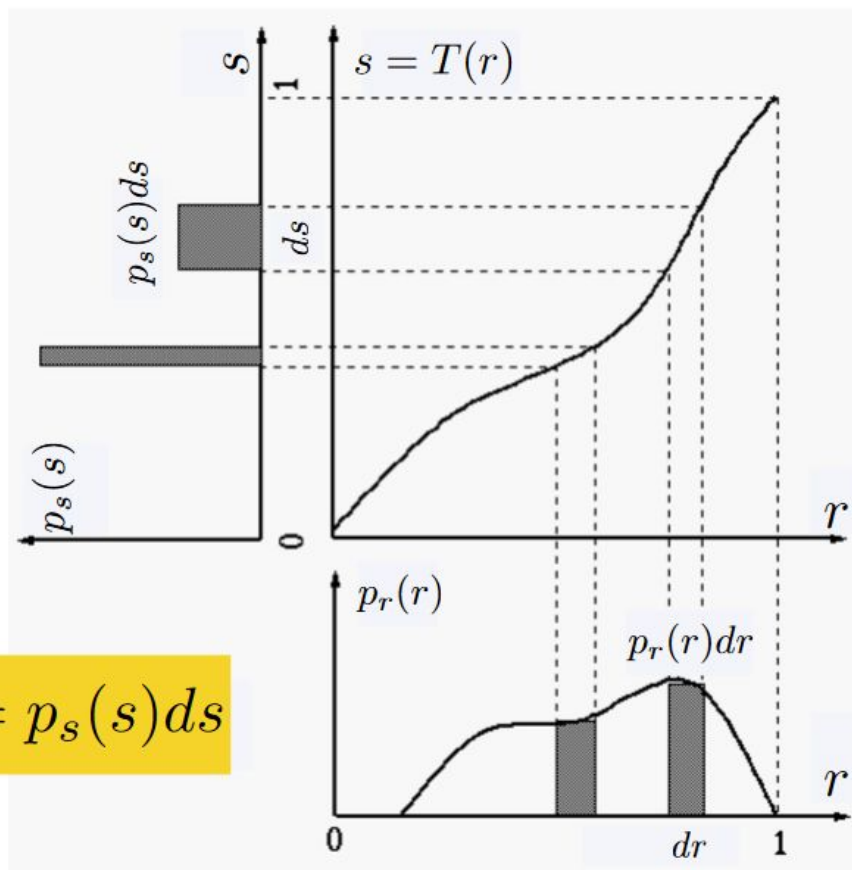
K. R. Jeripothula, J. Cai and J. Yuan, "QCCE: Quality constrained co-saliency estimation for common object detection," *2015 Visual Communications and Image Processing (VCIP)*, Singapore, 2015

Histogram equalization



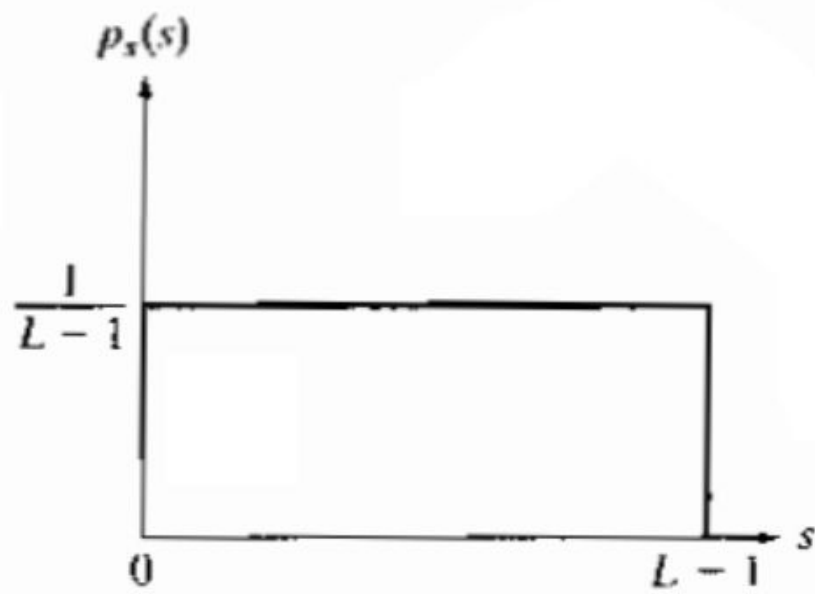
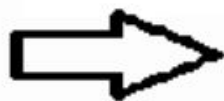
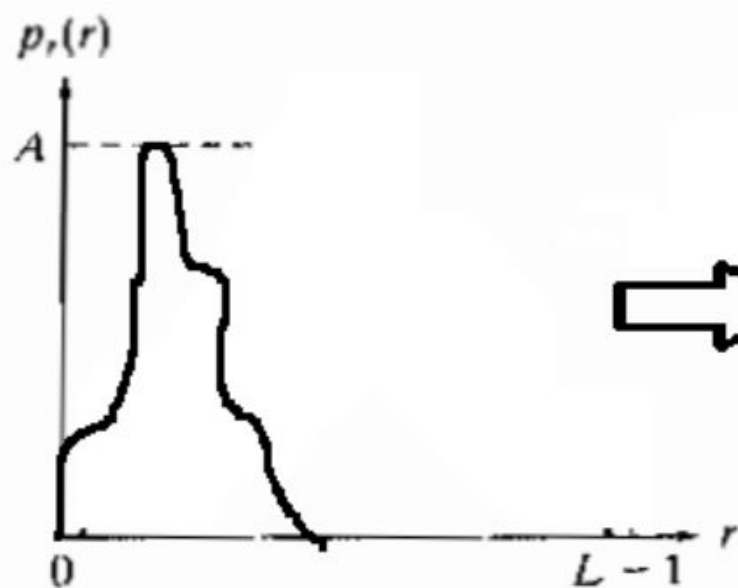
The main idea

Histogram equalization



$$p_r(r)dr = p_s(s)ds$$

The goal is to achieve a uniform pdf distribution as shown below:



$$s = T(r) = (L - 1) \int_0^r Pr(w) dw$$

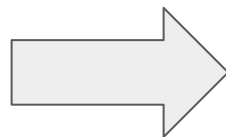
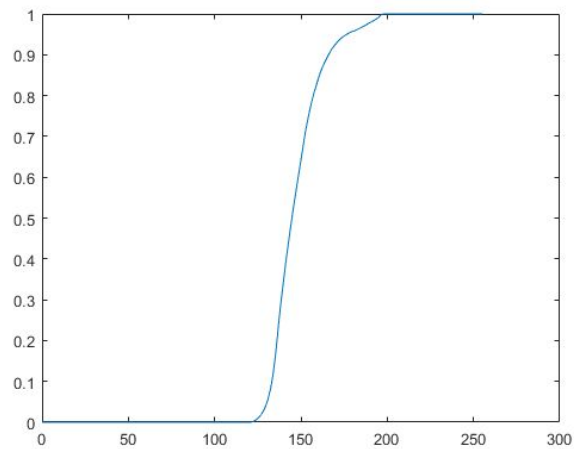
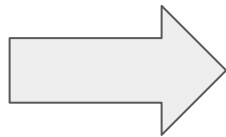
Fundamental Theorem of Calculus (FTC)

$$\text{FTC 1: } \frac{d}{dx} \int_a^x f(x) dx = f(x)$$

$$\begin{aligned} \frac{ds}{dr} &= \frac{d}{dr} T(r) \\ &= \frac{d}{dr} (L - 1) \int_0^r Pr(w) dw \\ &= (L - 1) Pr(r) \end{aligned}$$

$$\begin{aligned} Ps(s) &= Pr(r) \left| \frac{dr}{ds} \right| \\ &= Pr(r) \left| \frac{1}{(L - 1) Pr(r)} \right| \\ &= \frac{1}{(L - 1)} \end{aligned}$$

True for any transfer function



CDFs

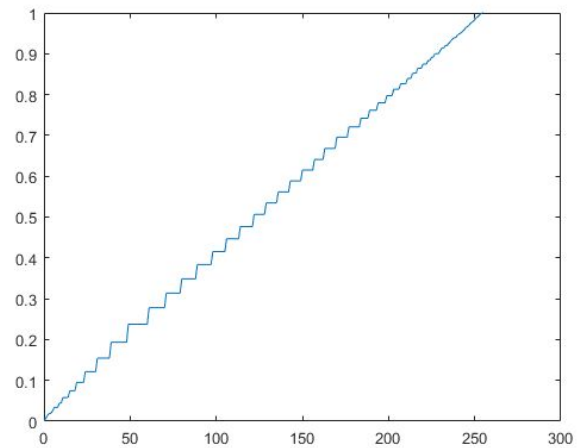
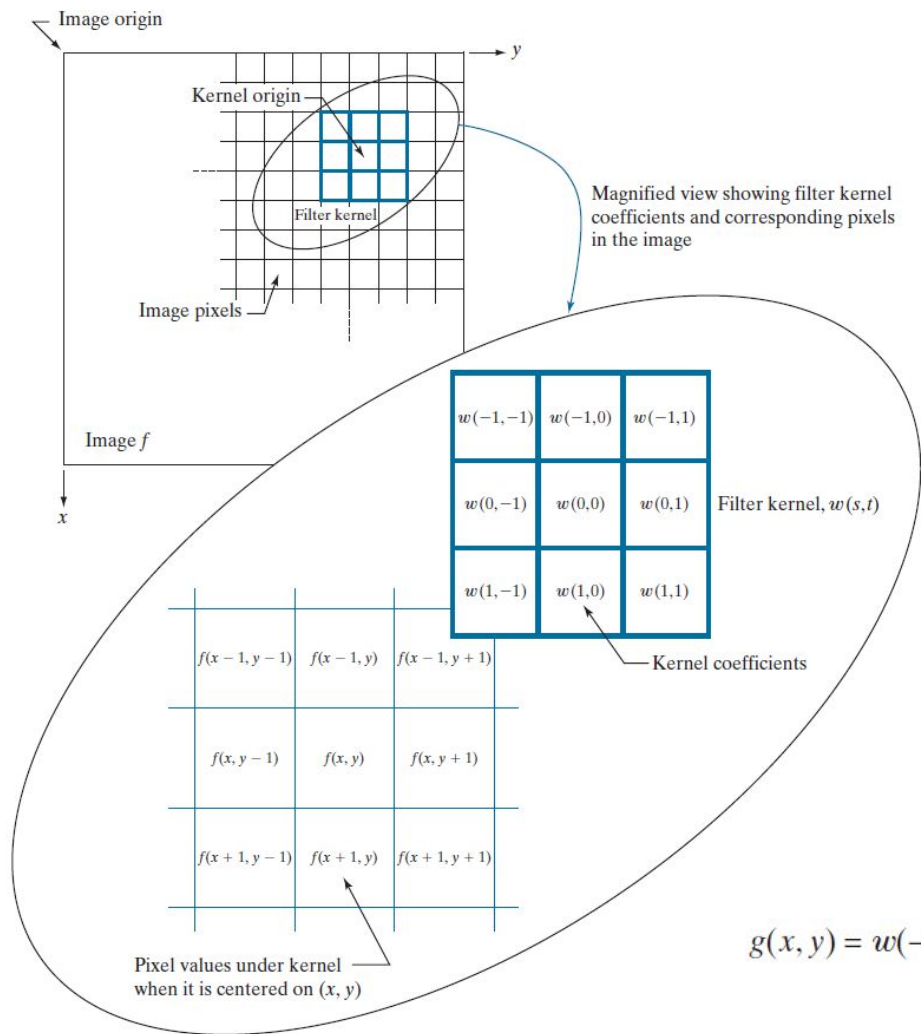


FIGURE 3.28

The mechanics of linear spatial filtering using a 3×3 kernel. The pixels are shown as squares to simplify the graphics. Note that the origin of the image is at the top left, but the origin of the kernel is at its center. Placing the origin at the center of spatially symmetric kernels simplifies writing expressions for linear filtering.



$$g(x, y) = w(-1, -1)f(x-1, y-1) + w(-1, 0)f(x-1, y) + \dots + w(0, 0)f(x, y) + \dots + w(1, 1)f(x+1, y+1)$$

For a kernel of size $m \times n$, we assume that $m = 2a + 1$ and $n = 2b + 1$

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)$$

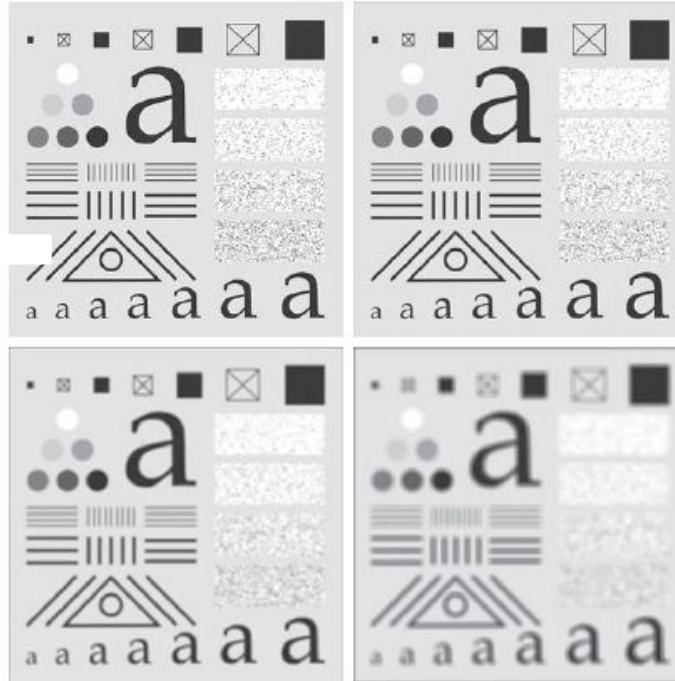
Since the operation is linear, we call it linear spatial filtering.

a	b
c	d

FIGURE 3.33

(a) Test pattern of size 1024×1024 pixels.

(b)-(d) Results of lowpass filtering with box kernels of sizes 3×3 , 11×11 , and 21×21 , respectively.



Box kernel:

$\text{Value_of_any_element} = 1 / \text{number_of_elements}$

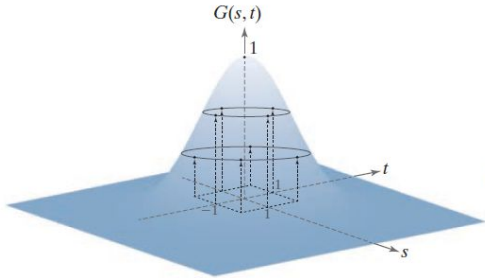
Gaussian Kernel

$$G(r) = Ke^{-\frac{r^2}{2\sigma^2}}$$

a b

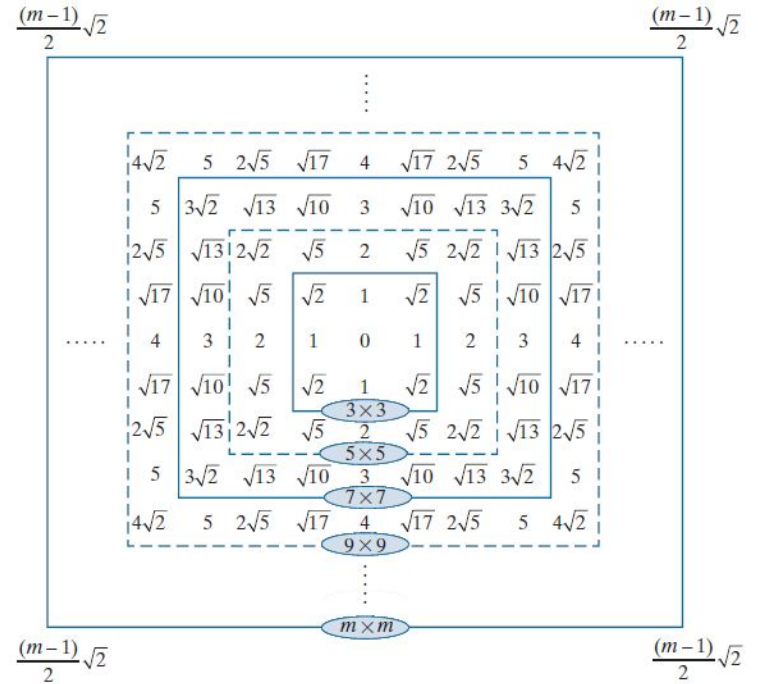
FIGURE 3.35

(a) Sampling a Gaussian function to obtain a discrete Gaussian kernel. The values shown are for $K = 1$ and $\sigma = 1$. (b) Resulting 3×3 kernel [this is the same as Fig. 3.31(b)].



$$\frac{1}{4.8976} \times$$

0.3679	0.6065	0.3679
0.6065	1.0000	0.6065
0.3679	0.6065	0.3679



Correlation vs Convolution

Spatial Correlation consists of moving the center of a kernel over an image, and computing the sum of products at each location.

The mechanics of spatial convolution are the same, except that the correlation kernel is rotated by 180° .

For symmetrical kernels, there is hardly any difference.

Correlation v/s Convolution

$$(w \star f)(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)$$

Correlation

$$(w \star f)(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x - s, y - t)$$

Convolution

image $f(x, y)$.

kernel w of size $m \times n$

$a = (m - 1)/2$, $b = (n - 1)/2$,

Some fundamental properties of convolution and correlation. A dash means that the property does not hold.

Property	Convolution	Correlation
Commutative	$f \star g = g \star f$	—
Associative	$f \star (g \star h) = (f \star g) \star h$	—
Distributive	$f \star (g + h) = (f \star g) + (f \star h)$	$f \star (g + h) = (f \star g) + (f \star h)$

$$w = w_1 \star w_2 \star w_3 \star \cdots \star w_Q$$

Separable Kernels

Also helpful when when kernels are separable.

A separable kernel is a matrix that can be expressed as the outer product of two vectors

$$w = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

is separable because it can be expressed as the outer product of the vectors

$$\mathbf{c} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \quad \text{and} \quad \mathbf{r} = \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}$$

That is,

$$\mathbf{c} \mathbf{r}^T = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = w$$

It turns out that the product of a column vector and a row vector is the same as the 2-D convolution of the vectors

$$w \star f = (w_1 \star w_2) \star f = (w_2 \star w_1) \star f = w_2 \star (w_1 \star f) = (w_1 \star f) \star w_2$$

Thus, the *computational advantage* of performing convolution with a separable, as opposed to a nonseparable, kernel is defined as

$$C = \frac{MNmn}{MN(m+n)} = \frac{mn}{m+n} \quad (3-44)$$

Examples [\[edit \]](#)

1. A two-dimensional [smoothing](#) filter:

$$\frac{1}{3} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} * \frac{1}{3} [1 \quad 1 \quad 1] = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

2. Another two-dimensional [smoothing](#) filter with stronger weight in the middle:

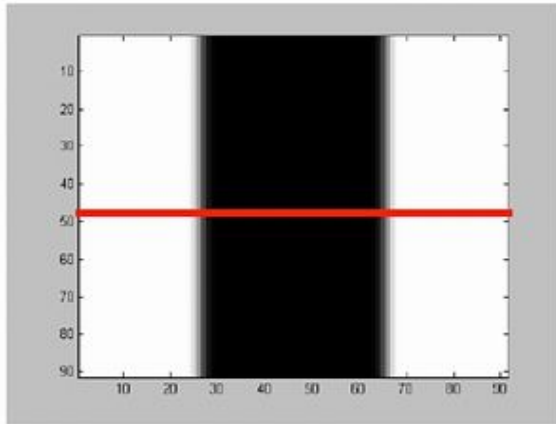
$$\frac{1}{4} \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} * \frac{1}{4} [1 \quad 2 \quad 1] = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

3. The [Sobel operator](#), used commonly for [edge detection](#):

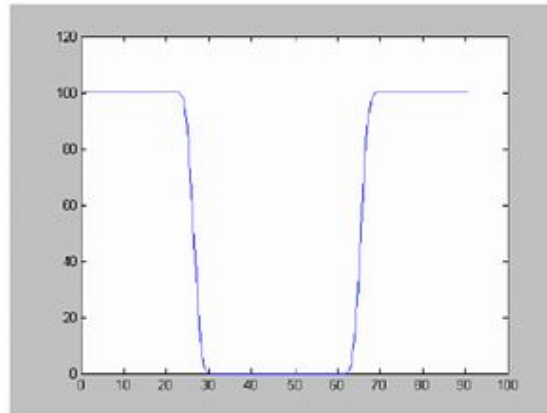
$$\begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} * [1 \quad 0 \quad -1] = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

Gradients

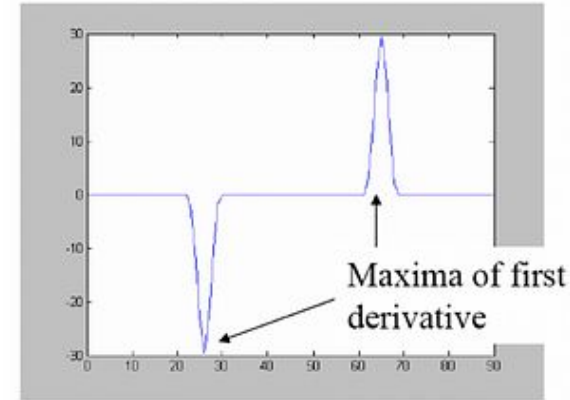
Useful for edge detection



Image



Intensities of pixels on the red line



1st derivative of pixel intensities

$$\frac{\partial f(x, y)}{\partial x} = \lim_{\varepsilon \rightarrow 0} \frac{f(x + \varepsilon, y) - f(x, y)}{\varepsilon}$$

Equation 1. Partial derivative of f with respect to x

$$\frac{\partial f(x, y)}{\partial x} \approx \frac{f(x + 1, y) - f(x, y)}{1}$$

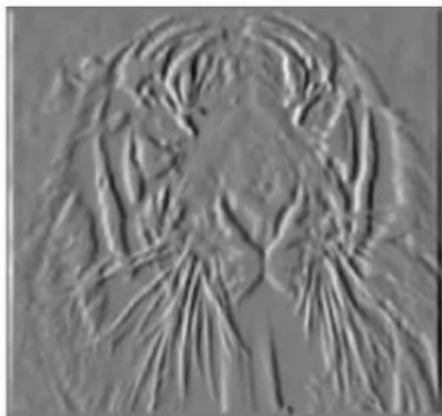
Equation 2. discretized Eq 1.



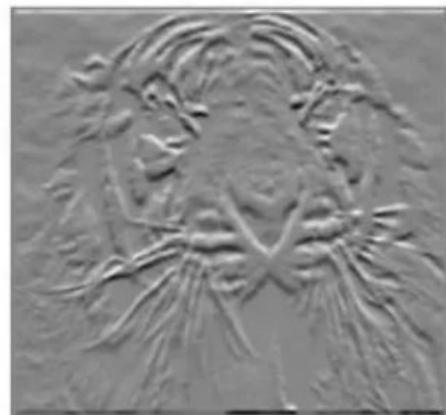
. Convolutional filter to compute the image gradient,



Image f



$$\frac{\partial f(x, y)}{\partial x}$$



$$\frac{\partial f(x, y)}{\partial y}$$

Fixing the center problem

- x-derivative filter: $[1 \ -1] \rightarrow [1 \ 0 \ -1]$
- y-derivative filter: $[1 \ -1]^T \rightarrow [1 \ 0 \ -1]^T$